

Hospital Asset Tracking and Smart Patient Flow Management System

A PROJECT REPORT

Submitted by

NASMIN M (611223114063)

SANGEETHA D (611223114083)

DHARSHANKUMAR S (611223114021)

ADITHYAN K G (611223114001)

in partial fulfilment for the award the degree of

BACHELOR OF ENGINEERING

IN

MECHANICAL ENGINEERING

KNOWLEDGE INSTITUTE OF TECHNOLOGY,

SALEM

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

BONAFIDE CERTIFICATE

Certified that this project report entitled “**Hospital Asset Tracking and Smart Patient Flow Management System**” is the bonafide work carried out by **NASMIN M, SANGEETHA D, DHARSHANKUMAR S, ADITHYAN K G** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Mechanical Engineering under Anna University, Chennai, during the academic year 2026–2027, under our supervision.

SIGNATURE

Dr. K. S. PRABHARAAN., M.E., Ph.D.
HEAD OF THE DEPARTMENT
PROFESSOR

DEPARTMENT OF MECHANICAL
ENGINEERING
KNOWLEDGE INSTITUTE OF
TECHNOLOGY
KIOT CAMPUS

KAKAPALYAM (PO), SALEM-
637504

SIGNATURE

Prof. S. NAVEENKUMAR, M.E., (Ph.D.)
SUPERVISOR
ASSISTANT PROFESSOR

DEPARTMENT OF MECHANICAL
ENGINEERING
KNOWLEDGE INSTITUTE OF
TECHNOLOGY
KIOT CAMPUS

KAKAPALYAM (PO), SALEM-
637504

ABSTRACT

Hospitals manage a continuous movement of patients, medical equipment, staff, and support services across multiple departments. When identification and tracking are performed manually, staff may spend significant time searching for equipment, updating registers, confirming patient movement, and reconciling records. These delays can reduce operational visibility and may contribute to misplaced assets or incomplete movement of histories. The proposed Hospital Asset Tracking and Smart Patient Flow Management System addresses this challenge through a prototype integration of radio-frequency identification (RFID), an ESP32 development module, Python processing, an Excel-based database, and a Streamlit dashboard.

The system uses RFID tags to represent patients and hospital assets such as wheelchairs, patient monitors, oxygen cylinders, and IV stands. An RC522 reader captures the unique identifier of a tag. The ESP32 receives the reader output through an SPI interface and transfers the UID to a Python application through serial communication. Python validates the UID, matches it against patient and asset records stored in an Excel workbook, appends a timestamped scan event, and exposes the updated information to a Streamlit dashboard. The dashboard presents identification details, current department, event history, scan counts, and basic patient-flow indicators.

The project demonstrates how embedded electronics, lightweight data management, and browser-based visualization can be combined to support a practical healthcare operations use case. The prototype is intended as an academic proof of concept rather than a certified clinical or safety-critical system. Its major contributions are the integration of a low-cost RFID identification layer, a transparent spreadsheet database, a Python processing layer, and a realtime dashboard that supports both asset visibility and patient

movement monitoring. Future work may include multi-location readers, cloud databases, automated alerts, role-based access control, privacy-preserving identifiers, and advanced flow analytics.

Keywords: RFID, RC522, ESP32, Internet of Things, Hospital Asset Tracking, Patient Flow, Python, Excel Database, Streamlit, Serial Communication, Smart Healthcare.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	TABLE OF CONTENTS	iv
	LIST OF TABLES	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
	LIST OF NOMENCLATURE	viii
1.	CHAPTER 1 - INTRODUCTION	1
1.1	Introduction	1
1.2	Background of the Project	1
1.3	Problem Statement	1
1.4	Existing System	2
1.5	Proposed System	2
1.6	Advantages of the Proposed System	2
1.7	Objectives of the Project	2

1.8	Scope of the Project	4
1.9	Significance of the Project	4
1.10	Organization of the Report	4
2.	CHAPTER 2 - LITERATURE REVIEW	5
2.1	Review Methodology	5
2.2	RFID in Operational Tracking	5
2.3	IoT and Embedded Healthcare Prototypes	5
2.4	Patient Flow Management	5
2.5	Asset Tracking in Hospitals	6
2.6	Interactive Data Dashboards	6
2.7	Spreadsheet-Backed Prototypes	6
2.8	Comparison of Approaches	6
2.9	Research Gap	6
2.10	Summary of Literature Review	7
3.	CHAPTER 3 - COMPONENTS, SPECIFICATION AND DESCRIPTION	8
3.1	Components	8
3.2	Introduction	10

3.3	Hardware Requirements	10
3.4	RC522 RFID Reader	10
3.5	ESP32 Dev Module	10
3.6	SPI Communication	10
3.7	Serial Communication	11
3.8	Python Data Processing	11
3.9	Excel Database	11
3.10	Streamlit Dashboard	11
3.11	Functional Requirements	11
3.12	Non-Functional Requirements	12
3.13	Development Environment	12
3.14	System Data Dictionary	12
4.	CHAPTER 4 - WORKING PRINCIPLE	13
4.1	Introduction	13
4.2	Overall System Architecture	13
4.3	Operational Workflow	15
4.4	Patient Flow Monitoring Design	16
4.5	Asset State Model	16

4.6	UID Matching Rules	16
4.7	Event Logging Design	17
4.8	Dashboard Design	17
4.9	Exception and Recovery Design	17
4.10	Security and Privacy Design	17
4.11	Chapter Summary	17
5.	CHAPTER 5 - COST ESTIMATION	18
5.1	Component-Wise Cost Estimation	18
5.2	Project Structure	18
5.3	ESP32 Firmware Logic	19
5.4	Python Serial Reader	19
5.5	Data Matching Implementation	19
5.6	Excel Event Logging	19
5.7	Duplicate Scan Handling	19
5.8	Streamlit Dashboard Implementation	20
5.9	Patient Flow View	20
5.10	Asset Tracking View	20
5.11	Configuration and Deployment	20

5.12	Implementation Summary	20
6.	CHAPTER 6 - RESULTS AND SYSTEM OUTPUTS	21
6.1	Introduction	21
6.2	Prototype Component Arrangement	21
6.3	Dashboard Layout	21
6.4	Successful Patient Identification	22
6.5	Successful Asset Identification	22
6.6	Scan History View	22
6.7	Patient Flow Output	22
6.8	Asset Utilization Output	22
6.9	Exception Output	22
6.10	Chapter Summary	22
7.	CHAPTER 7 - SCOPE OF FUTURE WORK	23
7.1	Introduction	23
7.2	Testing Methodology	23
7.3	Functional Test Cases	23
7.4	Communication Tests	24

7.5	Data and Storage Tests	24
7.6	Dashboard Tests	24
7.7	Integration Test Summary	24
7.8	Future Expansion	25
8.	CHAPTER 8 - APPENDICES	26
8.1	Introduction	26
8.2	Prototype Setup Checklist	27
8.3	Demonstration Dataset	28
8.4	Sample Event Log	28
8.5	Troubleshooting Guide	28
8.6	Risk Assessment	29
8.7	Operator User Guide	29
8.8	Validation Protocol for Future Multi Reader Deployment	29
8.9	Project Learning Outcomes	29
8.10	Final Appendix Summary	30
9.	CHAPTER 9 - CONCLUSION	31
9.1	Introduction	31

9.2	Project Outcome	31
9.3	Major Achievements	31
9.4	Significance of the Developed System	31
9.5	Limitations	31
9.6	Final Conclusion	31
10.	CHAPTER 10 - REFERENCES	31

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
Table 2.8	Comparison of manual, barcode, and RFID-based approaches	15
Table 3.1	Component specifications	17
Table 3.3	Hardware requirements	19
Table 3.9	Excel Database	20
Table 3.14	Data dictionary	21
Table 4.6	UID Matching Rules	22
Table 5.1	Component-Wise Cost Estimation	26
Table 5.2	Project Structure	26
Table 6.6	Scan History View	30
Table 7.3	Functional test cases	31
Table 7.8	Integration test summary	32
Table 8.11	Prototype Setup Checklist	35
Table 8.12	Demonstration Dataset	36
Table 8.13	Sample Event Log	36
Table 8.15	Risk Assessment	37

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
Figure 3.1	RC522 RFID reader module and pin reference	17
Figure 3.2	ESP32 development board	18
Figure 3.3	RFID reader and card demonstration kit	18
Figure 3.4	RFID key-fob tags	18
Figure 4.1	Overall system architecture	22
Figure 4.2	Operational workflow for a scan event	23
Figure 4.3	Patient flow monitoring concept	24
Figure 6.1	Example dashboard layout	29
Figure 6.2	Scan-event history view	30
Figure 7.1	Testing lifecycle	31

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CSV	Comma-Separated Values
ESP32	Espressif 32-bit Microcontroller Family
GUI	Graphical User Interface
IoT	Internet of Things
ISO	International Organization for Standardization
LED	Light-Emitting Diode
MFRC522	MIFARE contactless reader/writer IC
OPD	Outpatient Department
RFID	Radio-Frequency Identification
SPI	Serial Peripheral Interface
UID	Unique Identifier
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
USB	Universal Serial Bus

LIST OF NOMENCLATURE

Term / Notation	Description
RFID tag	A passive or active identifier attached to a patient wristband or hospital asset
Reader zone	The physical region in which a tag can be detected by a reader
UID	Identifier returned by the RFID tag and used as a lookup key
Scan event	A timestamped observation of a UID at a reader location
Dwell time	Elapsed time between entry and exit events for a location
Asset status	Operational state such as Available, In Use, Maintenance, or Missing
Patient flow	Movement of a patient through registered hospital departments
Streamlit session	Runtime context used to render an interactive dashboard
Serial frame	A line or message transmitted from ESP32 to the Python application
Data validation	Checks that ensure incoming values conform to expected formats

CHAPTER 1

INTRODUCTION

1.1 Introduction

A hospital is a complex service environment in which patients, clinical staff, diagnostic units, wards, pharmacies, and support services interact continuously. Alongside people, the hospital depends on mobile assets such as wheelchairs, monitors, oxygen cylinders, infusion stands, and emergency equipment. Visibility of these entities is essential for timely service delivery. However, many small and medium-sized facilities still rely on paper registers, verbal communication, and periodic manual checks to identify assets and record movement.

This project proposes a prototype smart healthcare operations system that connects low-cost RFID identification with embedded computing and a browser-based dashboard. The design is intentionally modular: the RFID layer captures identity, the ESP32 handles device-side communication, Python performs validation and matching, Excel stores structured records, and Streamlit presents information to users.

1.2 Background of the Project

Radio-frequency identification provides a convenient way to associate a physical object with a machine-readable identifier. In this project, the RFID identifier is not treated as a medical diagnosis or a replacement for clinical records; it is an operational key used to retrieve an approved record from a local database. The same pattern can be applied to patient movement and equipment movement, provided that privacy, consent, and access controls are considered.

The RC522 reader used in the prototype is based on the MFRC522 contactless reader/writer IC. The device supports 13.56 MHz contactless communication and ISO/IEC 14443 A/MIFARE/NTAG technologies [1]. The ESP32 is used as the controller because it can interface with sensors and peripherals and can communicate with a host computer through serial connectivity.

1.3 Problem Statement

Manual hospital tracking may result in delayed identification, incomplete movement records, misplaced equipment, duplicate data entry, and limited visibility of current patient or asset location. Staff may know that an asset exists but not whether it is available, in use, under maintenance, or assigned to another department. Similarly, a patient-flow record may be updated late or may depend on separate registers that are difficult to reconcile.

The problem addressed by this project is therefore the lack of a low-cost integrated prototype that can identify tagged entities, match them to structured records, preserve scan history, and display current operational information in a form that can be used for demonstration and future expansion.

1.4 Existing System

The common manual approach uses notebooks, printed registers, verbal confirmation, or spreadsheets updated after an event. Barcode-based approaches improve identification but require line-of-sight scanning and may become inconvenient when equipment is moving rapidly. Stand-alone RFID systems can identify tags without direct line of sight but may not provide a unified history or user-friendly dashboard.

The prototype developed in this project focuses on integration rather than replacing a full hospital information system. It demonstrates a transparent data path from physical scan to dashboard record and makes the event history available for review.

1.5 Proposed System

The proposed system consists of RFID tags, one or more RC522 readers, an ESP32 Dev Module, serial communication, a Python application, an Excel workbook, and a Streamlit dashboard. Each tag is mapped to a record containing an entity type, name or asset label, department, status, and other non-sensitive demonstration fields. When a scan occurs, a new event is created with the UID, timestamp, reader or department, and operator or session information.

The dashboard can show the latest scan, matched record, event history, scan count, department summary, and asset status. The same architecture can later be connected to multiple readers, a cloud database, alerting services, or an enterprise integration layer.

1.6 Advantages of the Proposed System

The proposed approach offers an observable and low-cost prototype that is easy to demonstrate and modify. It brings together embedded programming, communication, data processing, and visualization in one system. Because the Excel workbook is human-readable, the project team can inspect records and validate test cases without specialized database administration tools. Streamlit further reduces the effort needed to present interactive tables and metrics in a browser [3].

1.7 Objectives of the Project

The objectives of the project are to design and implement an RFID-based identification mechanism, collect tag UIDs using an ESP32 and RC522 reader, transmit scan information to a Python application, match the UID against patient and asset records, maintain scan time and scan count fields, and present the result through a Streamlit dashboard.

1.8 Scope of the Project

The scope is limited to an academic prototype that demonstrates identification and operational tracking. It covers patient and asset records, scan-event logging, basic department flow, dashboard visualization, and error messages for unknown or invalid UIDs. It does not provide clinical decision support, emergency dispatch, medical diagnosis, billing, or a certified patient identification workflow.

1.9 Significance of the Project

The project is significant because it demonstrates how an engineering team can convert a realworld hospital operations problem into a complete proof of concept. It provides a foundation for learning system integration, modular design, data validation, visualization, and responsible handling of healthcare-related information.

1.10 Organization of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 reviews related approaches. Chapter 3 describes the hardware, software, and data requirements. Chapter 4 presents the architecture and design. Chapter 5 explains implementation. Chapter 6 records representative outputs. Chapter 7 discusses testing and performance observations. Chapter 8 identifies enhancements, Chapter 9 concludes the report, and Chapter 10 lists references.

CHAPTER 2

LITERATURE REVIEW

2.1 Review Methodology

The review focused on technologies relevant to the prototype: RFID identification, embedded controllers, serial and SPI communication, IoT-oriented data collection, spreadsheet-backed records, and interactive dashboards. Official manufacturer documentation and project documentation were prioritized for technical facts, while general system comparisons were treated as design analysis rather than clinical evidence.

2.2 RFID in Operational Tracking

RFID systems use radio communication to identify tagged objects. A reader creates an interrogation field and receives a response from a compatible tag. Compared with manual entry, an RFID workflow can reduce repetitive typing and can associate an event with a machinereadable identifier. The reliability of an installation depends on tag orientation, reader position, antenna characteristics, interference, read range, and the quality of event de-duplication.

The MFRC522 product documentation describes the IC as a highly integrated reader/writer for contactless communication at 13.56 MHz and identifies support for ISO/IEC 14443 A, MIFARE, and NTAG technologies [1]. These capabilities make it suitable for a small-scale educational prototype, while its limitations must be considered before any production deployment.

2.3 IoT and Embedded Healthcare Prototypes

IoT prototypes commonly separate sensing, edge processing, transport, storage, and visualization. In this project, the RFID reader and ESP32 form the sensing and edge layer, serial communication forms the local transport layer, Python provides processing, Excel provides storage, and Streamlit provides visualization. The division helps isolate faults and enables each layer to be tested independently.

Espressif maintains technical documents and programming guidance for ESP32 devices and related development frameworks [2]. The exact board variant and firmware configuration should be recorded during deployment

because pin assignments, power requirements, and available peripherals can differ across boards.

2.4 Patient Flow Management

Patient flow management concerns the movement of patients through registration, consultation, diagnostics, pharmacy, ward, and discharge processes. A tracking prototype can support visibility by recording transitions between defined locations. The value of the event log depends on clear operational rules: every location should have an identifier, scan events should be timestamped consistently, duplicate taps should be handled, and exceptional movement should be explainable.

2.5 Asset Tracking in Hospitals

Mobile assets are difficult to manage when they are shared among departments. A useful asset record typically includes an asset ID, category, current status, assigned department, maintenance state, last scan time, and optional owner or custodian. The prototype models these fields while keeping the data structure simple enough to be maintained in Excel.

2.6 Interactive Data Dashboards

Interactive dashboards allow users to filter, sort, and inspect operational records without opening the underlying data file. Streamlit is an open-source Python framework intended for dynamic data applications and is documented as a way to build data apps with relatively little code [3]. In this project, the dashboard acts as a visual layer over the processed workbook and scan log.

2.7 Spreadsheet-Backed Prototypes

Excel is not a substitute for a hospital-grade transaction database, but it is useful in early prototypes because it is familiar, inspectable, and easy to exchange. A spreadsheet-backed design should still use unique identifiers, controlled columns, validation rules, and an append-only event sheet wherever possible. This project treats Excel as a prototype database that can later be migrated to SQLite, PostgreSQL, or a managed cloud database.

2.8 Comparison of Approaches

The following comparison summarizes the design rationale for choosing RFID over manual registers and line-of-sight barcodes in this prototype.

Table 2.8 Comparison of Approaches

Criterion	Manual register	Barcode	RFID prototype
Identification	Human-readable entry	Line-of-sight scan	Contactless UID scan
Data entry	High manual effort	Moderate	Low after mapping
History	Often incomplete	Possible with software	Built into scan-event log
Asset visibility	Periodic or verbal	Requires scanning	Can be placed at checkpoints
Cost and complexity	Low cost, low automation	Moderate	Moderate prototype complexity
Limitations	Errors and delays	Orientation and line of sight	Read range, interference, privacy, duplicate events

2.9 Research Gap

Many demonstrations focus on either RFID hardware or dashboard software in isolation. The project gap addressed here is the integrated educational workflow that links a physical tag to a patient or asset record, stores a timestamped event, and presents both entity status and department flow in one application.

2.10 Summary of Literature Review

The review supports a layered architecture. RFID is suitable for identification, ESP32 provides a compact embedded interface, Python is appropriate for data transformation, Excel is acceptable for a transparent proof of concept, and Streamlit can provide an interactive visualization layer. The review also establishes that a production system would require stronger security, privacy, availability, auditability, and clinical governance than the prototype provides.

CHAPTER 3

COMPONENTS, SPECIFICATION AND DESCRIPTION

3.1 Components

The prototype consists of RFID tags, the RC522 RFID reader module, the ESP32 Dev Module, a USB serial connection, a host computer, an Excel workbook, Python processing modules, and a Streamlit dashboard. The hardware and software components are selected to support a transparent, low-cost hospital operations prototype.

Table 3.1 Components

Component	Specification / role	Application in project
RFID tag	13.56 MHz contactless identifier	Patient wristband or asset label
RC522 reader	MFRC522-based reader/writer module	Reads tag UID
ESP32 Dev Module	Embedded controller with SPI and serial interfaces	Captures and transmits UID
Python	Processing and integration language	Matching, logging, aggregation
Excel workbook	Prototype structured storage	Patients, assets, departments, scan log
Streamlit	Python dashboard framework	Real-time visualization and filtering

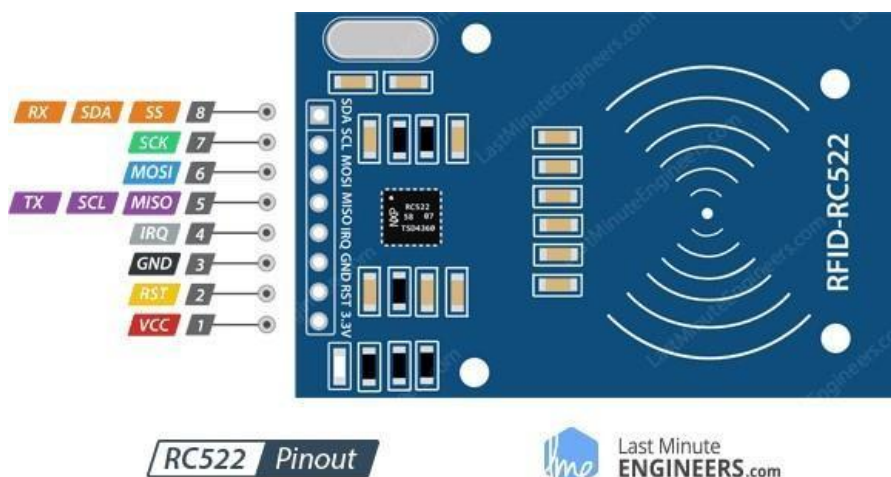


Figure 3.1 RC522 RFID reader module and pin reference.

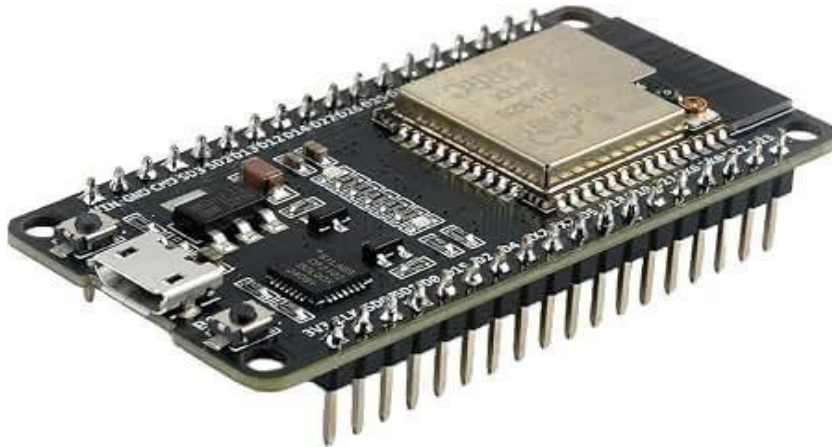


Figure 3.2 ESP32 development board used as the embedded controller.



Figure 3.3 RFID reader and card demonstration kit.



Figure 3.4 Example RFID key-fob tags for asset identification.

3.2 Introduction

This chapter describes the components selected for the project and translates the operational problem into hardware, software, data, and user requirements. The selection emphasizes accessibility, low cost, ease of learning, and a clear path toward future expansion.

3.3 Hardware Requirements

The hardware consists of RFID tags, an RC522 reader module, an ESP32 Dev Module, jumper wires, a breadboard or mounting surface, a USB cable, and a host computer. Optional status of LEDs or a buzzer may be used to indicate a successful or failed read. The reader and controller should be powered according to their specified voltage requirements, and common ground must be maintained.

Table 3.3 Hardware Requirements

Component	Purpose	Key consideration
RFID tag	Represents a patient wristband or asset	Unique UID; physical attachment
RC522 reader	Reads compatible contactless tags	Antenna alignment and read range
ESP32 Dev Module	Collects UID and transmits data	Firmware, power, serial settings
USB cable	Power and serial link to host	Stable connection
Host computer	Runs Python and Streamlit	Python environment and port access
Mounting accessories	Hold reader at checkpoint	Consistent reader placement

3.4 RC522 RFID Reader

The RC522 module is based on the MFRC522 contactless reader/writer IC. It is designed for 13.56 MHz contactless communication and supports ISO/IEC 14443 A/MIFARE/NTAG technologies [1]. The reader is connected to the ESP32 through SPI in the prototype. The software must initialize the bus, select the reader, detect a new card, read the serial number, and halt the card after a successful read.

3.5 ESP32 Dev Module

The ESP32 acts as the local controller between the RC522 reader and Python. It executes firmware that initializes SPI, polls the reader, formats the UID, and sends a serial message. The use of a controller separates timing-sensitive reader operations from Python processing and keeps the hardware interface extensible. Espressif documentation should be consulted for the specific board, GPIO assignment, peripheral configuration, and programming framework [2].

3.6 SPI Communication

SPI is a synchronous serial interface commonly used to connect a controller to a peripheral. The RC522 connection requires a clock, data lines, chip select, and reset control in addition to power and ground. The exact GPIO

mapping is board-dependent and should be recorded in the firmware source and project documentation.

3.7 Serial Communication

After the ESP32 obtains the UID, it sends a text frame through USB serial. A simple frame can contain a normalized hexadecimal UID followed by a newline. Python reads the port, strips whitespace, validates the character set and length, and passes the value to the matching function.

A text protocol is appropriate for the prototype because it is easy to observe and debug.

3.8 Python Data Processing

Python is used for serial input, normalization, matching, event creation, aggregation, and dashboard preparation. The processing layer should separate hardware input from business logic so that test cases can inject known UIDs without requiring a physical scan.

3.9 Excel Database

The workbook may contain sheets such as Patients, Assets, ScanLog, Departments, and Configuration. Each patient or asset record has a stable RFID UID. The ScanLog sheet is append-oriented and stores one row per accepted scan. A backup copy should be created before modifying the workbook, and concurrent editing should be avoided in the prototype.

Table 3.9 Excel Database

Sheet	Purpose	Representative columns
Patients	Patient identity mapping	UID, Patient_ID, Name_Label, Department, Status
Assets	Equipment mapping	UID, Asset_ID, Category, Department, Status
ScanLog	Event history	Timestamp, UID, Entity_Type, Location, Result

Departments	Location reference	Department_ID, Department_Name, Reader_ID
Configuration	System settings	Serial_Port, Baud_Rate, Duplicate_Window

3.10 Streamlit Dashboard

Streamlit provides the browser-facing interface for the prototype. The dashboard may include summary metrics, the latest scan card, patient-flow tables, asset-status tables, and filters. Streamlit documentation describes the framework as an open-source Python framework for building dynamic data apps [3].

3.11 Functional Requirements

The system shall read a UID from a connected reader, identify whether it belongs to a patient or an asset, display a clear result, record the event time, update the scan count, and show historical records. It shall also distinguish unknown UIDs, malformed input, duplicate events, missing records, unavailable serial ports, and workbook access errors.

3.12 Non-Functional Requirements

The prototype should be understandable, maintainable, responsive for small datasets, and recoverable after common input errors. The dashboard should avoid exposing unnecessary personal information. The application should provide visible feedback and should not silently discard an event.

3.13 Development Environment

A typical environment includes Python 3.x, a virtual environment, a serial communication package, pandas, openpyxl, Streamlit, and the ESP32 development toolchain. Version pinning is recommended for reproducibility. The report records the logical dependencies rather than assuming that one exact operating system or IDE is required.

3.14 System Data Dictionary

The data dictionary defines the meaning and format of each operational field so that hardware, processing, and dashboard layers use consistent names.

Table 3.14 System Data Dictionary

Field	Type	Description	Validation
UID	Text	RFID identifier used as key	Hexadecimal string, non-empty
Entity_Type	Text	Patient or Asset	Controlled vocabulary
Entity_ID	Text	Internal record identifier	Unique within entity type
Location	Text	Current or scanned department	Must match department list
Timestamp	DateTime	Time of accepted event	Valid local timestamp
Status	Text	Current operational state	Controlled vocabulary
Scan_Count	Integer	Number of accepted scans	Non-negative

CHAPTER 4

WORKING PRINCIPLE

4.1 Introduction

System design converts the requirements into modules, interfaces, data flows, and operating rules. The design is intentionally layered so that the reader, controller, processing application, database, and dashboard can be developed and tested separately.

4.2 Overall System Architecture

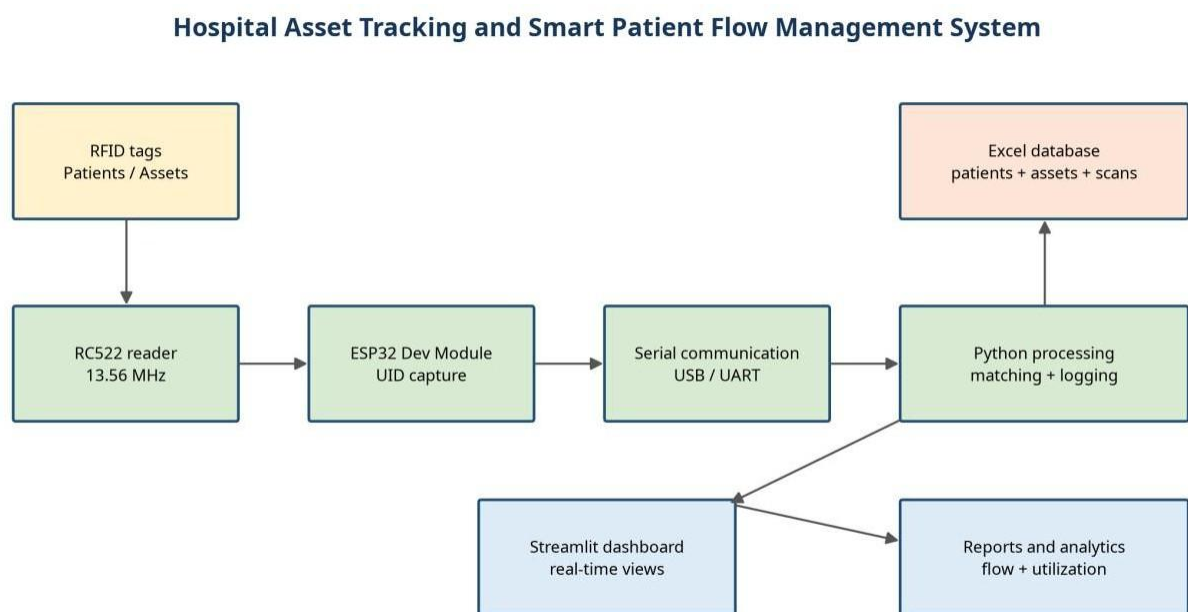


Figure 4.1 Overall system architecture.

The architecture begins with a physical RFID tag. The RC522 reader detects the tag, and the ESP32 captures its UID using SPI. The UID is sent to the Python application over serial communication. Python performs validation and matching against the Excel workbook, appends a scan event, and prepares data for Streamlit. The dashboard then presents the latest state and historical information.

4.2.1 Identification Layer

The identification layer consists of the tag and the reader. A tag is assigned to a demonstration of patient or asset record during setup. The

assignment must be controlled because a UID by itself has no meaningful business interpretation until it is mapped to an authorized record.

4.2.2 Embedded Layer

The embedded layer initializes the reader, polls for cards, reads the UID, formats it consistently, and transmits it. It should avoid sending repeated events continuously while a tag remains in the reader field. A debounce or duplicate window is therefore applied either in firmware or in Python.

4.2.3 Processing and Storage Layer

The Python layer is responsible for data integrity. It normalizes UID strings, checks the entity tables, creates an event record, updates the latest known location, and communicates a result to the dashboard. The Excel workbook acts as the persistence layer for the prototype.

4.2.4 Presentation Layer

The Streamlit layer renders metrics and tables. It should show enough context for the operator to understand the event, including whether the UID was recognized, which entity was found, where the scan occurred, and when the event was recorded.

4.3 Operational Workflow

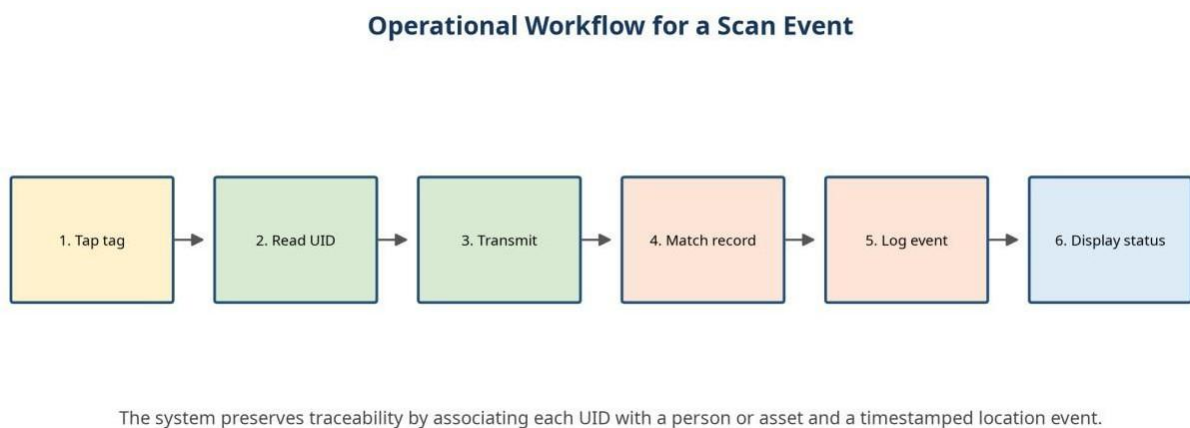


Figure 4.2 Operational workflow for a scan event.

The operational workflow starts when a tagged patient or asset is brought within the reader zone. The reader returns a UID, the ESP32 transmits it, and Python performs a lookup. If the UID is known, the system creates a successful scan event and updates the record. If it is unknown, the system

records an exception result and displays a message requesting registration or verification.

4.4 Patient Flow Monitoring Design

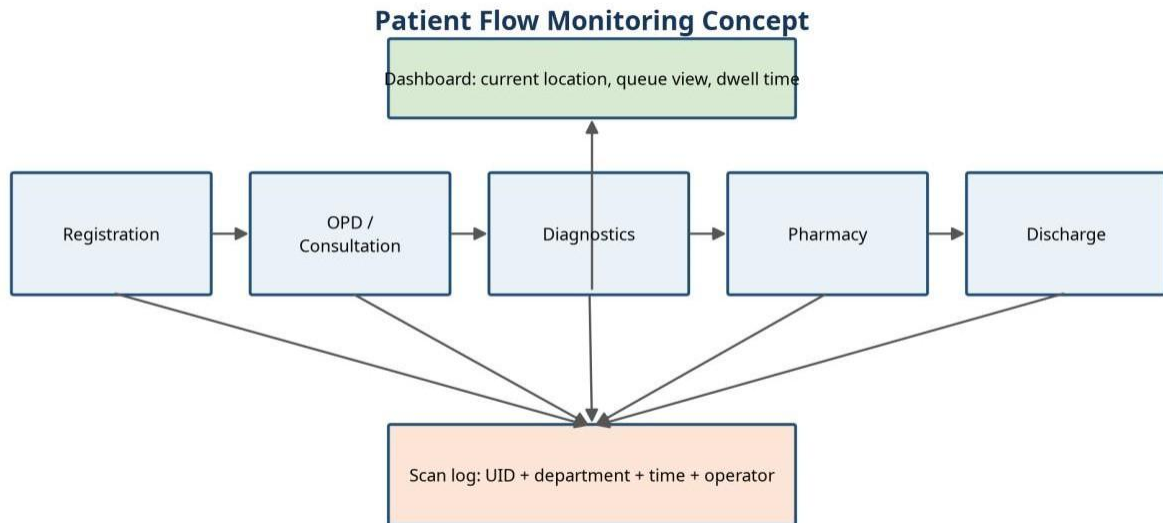


Figure 4.3 Patient flow monitoring concept.

Patient-flow monitoring is implemented as a sequence of department observations. Each observation consists of a UID, a location, and a timestamp. A dashboard can derive the current department from the most recent accepted event and can calculate approximate dwell time when an entry and exit rule is defined. The prototype does not infer clinical urgency or diagnose a patient; it only visualizes registered movement events.

4.5 Asset State Model

An asset can be represented by a small state machine: Available, Assigned, In Use, Under Maintenance, and Not Located. A successful scan may update the current location and assigned state, while a maintenance action may update the state without a scan. The design keeps status changes explicit so that future workflows can include authorized state transitions.

4.6 UID Matching Rules

Matching is case-insensitive after normalization. Whitespace and separators are removed or standardized. A UID must match exactly one record in the combined patient and asset mapping. If it matches none, the result is

Unknown. If it matches more than one record, the result is Data Conflict, and the event is not treated as a valid identification.

Table 4.6 UID Matching Rules

Input condition	System result	Dashboard message
Known patient UID	Create patient scan event	Patient identified; show current department
Known asset UID	Create asset scan event	Asset identified; show status and location
Unknown UID	Create exception event	Tag not registered
Malformed UID	Reject or flag event	Invalid UID format
Duplicate within time window	Suppress duplicate	Recent scan already recorded
Conflicting records	Do not update state	Data conflict; administrator review

4.7 Event Logging Design

Each scan event should be append-oriented. Suggested fields include Event_ID, Timestamp, UID, Entity_Type, Entity_ID, Reader_ID, Department, Operator, Result, and Notes. The event log provides traceability and supports later analysis of scans per department, asset utilization, and patient movement.

4.8 Dashboard Design

The dashboard is organized into summary metrics, latest events, patient-flow views, asset-status view, and event history. Filters may include date, entity type, department, asset category, and event result. A refresh action should reload the workbook safely and should report errors instead of presenting stale data as current.

4.9 Exception and Recovery Design

The application should handle a missing serial port, disconnected ESP32, empty workbook, missing columns, duplicate UIDs, and file permission errors. Each error should be visible and actionable. For example, a serial-port error should tell the operator to check the cable and configured port, while a workbook schema error should identify the missing column.

4.10 Security and Privacy Design

The prototype should use demonstration data and avoid unnecessary personal identifiers. In a real deployment, patient records would require authentication, authorization, audit logs, encryption in transit and at rest, retention policies, and governance. RFID UIDs should not be displayed as if they were medical information. Access to the dashboard should be limited to authorized staff, and exported files should be protected.

4.11 Chapter Summary

The design integrates physical identification, embedded communication, Python validation, spreadsheet storage, and interactive visualization. The architecture provides a clear path from tag scan to dashboard output while explicitly separating prototype capabilities from production healthcare requirements.

CHAPTER 5

COST ESTIMATION

5.1 Component-Wise Cost Estimation

The prototype is designed to remain affordable for academic demonstration. The values below are indicative planning estimates only; actual prices depend on supplier, region, quality, shipping, and availability.

Table 5.1 Component-Wise Cost Estimation

Item	Quantity	Indicative unit cost	Indicative total cost
RC522 RFID reader module	1	₹150	₹150
ESP32 Dev Module	1	₹450	₹450
RFID cards / key fobs	10	₹30	₹300
Breadboard and jumper wires	1 set	₹250	₹250
USB cable and power accessories	1 set	₹200	₹200
Host computer and software	1	Existing resource	—
Estimated prototype hardware total			₹1,350

The cost estimate excludes the host computer because it is assumed to be available during development. A production deployment would additionally require readers at multiple checkpoints, mounting hardware, network infrastructure, backup systems, access control, database hosting, cybersecurity review, and maintenance support.

5.1 Introduction

Implementation converts the system design into firmware, Python modules, workbook sheets, and dashboard components. The project was developed incrementally: first the reader was tested, then UID transmission, then record matching, then event logging, and finally dashboard presentation.

5.2 Project Structure

A modular project structure supports maintenance and testing. Hardware code, processing logic, data files, and presentation code are kept distinct.

Table 5.2 Project Structure

Folder / file	Purpose
firmware/esp32_rfid.ino	ESP32 firmware for RC522 and serial output
data/patients.xlsx	Prototype workbook containing mapping and logs
src/serial_reader.py	Serial connection and frame parsing
src/matcher.py	UID normalization and record lookup
src/logger.py	Event creation and workbook updates
app.py	Streamlit dashboard entry point
config.yaml or. env	Local configuration values
README.md	Setup and operating instructions

5.3 ESP32 Firmware Logic

The firmware initializes serial communication and the SPI bus, configures the RC522 reader, and repeatedly checks for a new card. After reading the UID, it converts bytes to a consistent uppercase hexadecimal string and prints one complete frame. The firmware also stops communication with the card before returning to the polling loop.

A conceptual serial frame is shown below:

UID=04A1B2C3D4E5; READER=MAIN_GATE;

5.4 Python Serial Reader

The Python serial reader opens the configured port and baud rate, reads complete lines, and forwards each line to a parser. The parser removes the UID label, trims whitespace, rejects unexpected characters, and returns a normalized value. Unit testing can call the parser with strings without requiring hardware.

5.5 Data Matching Implementation

The matching layer loads the Patients and Assets sheets into data frames, verifies required columns, normalizes UUIDs, and searches for an exact match. The result is represented as a structured object containing the result type, entity type, entity ID, label, department, and status.

Keeping this result structured prevents the dashboard from depending on fragile string parsing.

5.6 Excel Event Logging

The logger creates a timestamp using the configured local time convention and appends a row to ScanLog. It also updates the latest department or location in the corresponding entity sheet if the operational rule allows such an update. Before writing, the application checks that the workbook exists, is writable, and contains the expected sheets.

5.7 Duplicate Scan Handling

A tag can remain in a reader field for several polling cycles. To prevent inflated counts, the processing layer compares the new UUID and reader with the most recent accepted event. If the difference is less than the configured duplicate window, the event may be marked to Duplicate or suppressed according to the selected policy.

5.8 Streamlit Dashboard Implementation

The dashboard reads processed data and renders metrics, data frames, filters, and status messages. The latest scan is displayed prominently, while historical records are shown in a table. The page should refresh after an accepted event and should make the data timestamp visible so that the operator can distinguish a current view from an old one.

5.9 Patient Flow View

The patient-flow view groups for the most recent accepted scan per patient and displays current department, last scan time, and movement status. A department summary counts patients whose latest event belongs to each department. These values are descriptive and depend on the completeness of scan events.

5.10 Asset Tracking View

The asset view groups records by category and status and shows the latest known department. Users can filter wheelchairs, monitors, oxygen cylinders, or IV stands. This helps demonstrate how the same infrastructure can support both patient movement and equipment visibility.

5.11 Configuration and Deployment

The serial port, baud rate, workbook path, duplicate window, and reader ID should be configurable rather than hard-coded. A local development setup can run the dashboard with a Streamlit command, while a future deployment may use a service manager, container, or managed platform. Credentials and secrets should not be placed in the workbook or source code.

5.12 Implementation Summary

The implementation demonstrates a complete vertical slice: hardware input is converted into a UID, the UID is matched to a record, the event is logged, and the dashboard presents the result. The separation of concerns makes it possible to replace Excel with a relational database or to replace serial communication with a network protocol later.

CHAPTER 6

RESULTS AND SYSTEM OUTPUTS

6.1 Introduction

This chapter describes the representative outputs expected from the prototype. Since the system is an academic prototype, the results emphasize functional integration and observable behavior rather than claiming clinical performance.

6.2 Prototype Component Arrangement

The prototype can be arranged with the RC522 reader connected to the ESP32 on a breadboard or mounted at a checkpoint. RFID cards or key fobs are assigned to demonstration patients and assets. The ESP32 is connected to the host computer through USB, and the Python application runs alongside the Streamlit dashboard.

6.3 Dashboard Layout

A practical dashboard layout contains a title, connection status, latest event card, summary metrics, patient-flow table, asset-status table, and scan-event history. The layout should prioritize the latest event and exceptions because these are the most operationally relevant outputs.

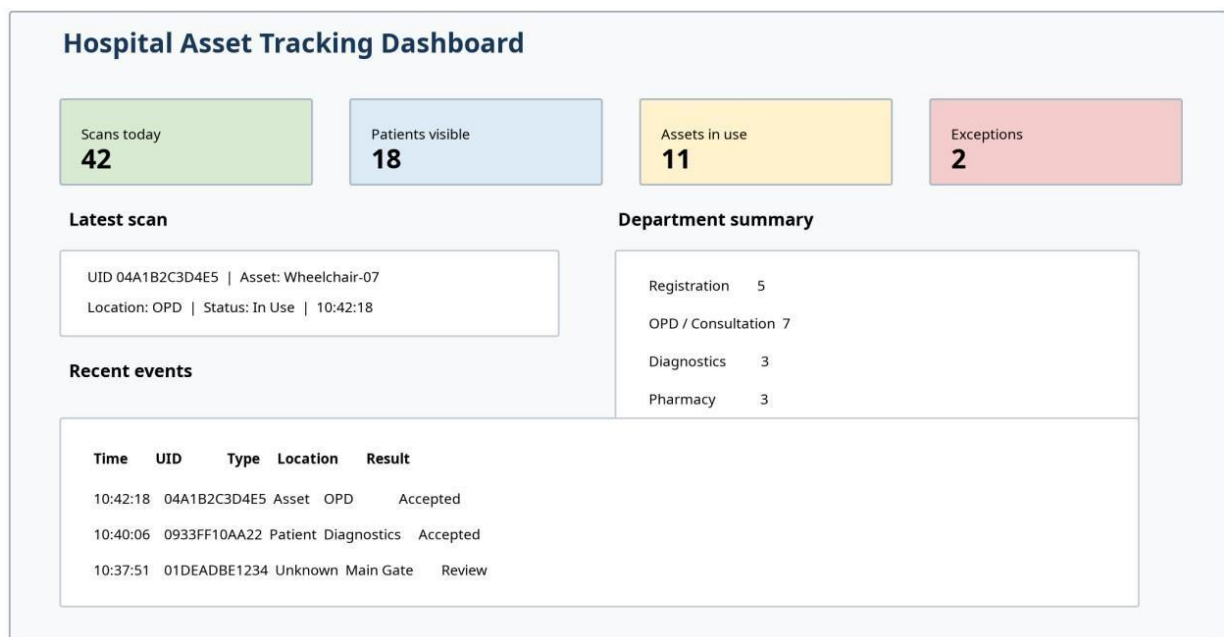


Figure 6.1 Representative dashboard layout for the prototype.

6.4 Successful Patient Identification

When a registered patient tag is scanned, the dashboard should identify the record, show the current department, display the scan time, and append a row to the event history. The output confirms the end-to-end path from RFID reader to visual result.

6.5 Successful Asset Identification

When a tagged asset is scanned, the dashboard should show the asset category, asset label, current status, and location. For example, a wheelchair may be represented as In Use at OPD, while an oxygen cylinder may be represented as Available in the equipment store.

6.6 Sample Event Log

The scan history view provides traceability. Each row should contain enough context to distinguish a patient scan from an asset scan and to identify the reader or department that generated the event.

Table 6.6 Sample Event Log

Time	UID	Type	Location	Result
10:42:18	04A1B2C3D4E5	Asset	OPD	Accepted
10:40:06	0933FF10AA22	Patient	Diagnostics	Accepted
10:37:51	01DEADBE1234	Unknown	Main Gate	Review
10:35:24	088877665544	Asset	Equipment Store	Accepted

6.7 Patient Flow Output

The patient-flow view can display the latest department per registered patient and the time since the last event. A complete flow view depends on scanning at each defined checkpoint. If a patient moves without a scan, the dashboard should not claim to know the actual location; it should show the last recorded location and timestamp.

6.8 Asset Utilization Output

The asset view can summarize available, assigned, in-use, maintenance, and not-located assets. A utilization indicator may be calculated as the number of assets in use divided by the number of operational assets, provided that the dataset and status rules are clearly defined.

6.9 Exception Output

Unknown UUIDs, duplicate scans, missing records, and connection errors should be displayed as exceptions. The system should not silently convert an unknown UUID into a valid patient or asset record.

6.10 Chapter Summary

The result of the project is an integrated prototype that produces visible, timestamped identification and movement outputs. Its value lies in functional connectivity and traceability; formal clinical validation, large-scale performance testing, and production security remain outside the current scope.

CHAPTER 7

SCOPE OF FUTURE WORK

7.1 Introduction

Testing was planned at hardware, communication, processing, storage, interface, and integration levels. The test strategy emphasizes observable acceptance criteria and repeatable inputs.

7.2 Testing Methodology

The system is tested in layers. Hardware tests confirm that the RC522 can detect a tag and that the ESP32 can read the UID. Communication tests confirm that a complete serial frame reaches Python. Data tests confirm matching and logging. Interface tests confirm that the dashboard reflects the new event. Negative tests confirm that errors are visible and do not corrupt records.

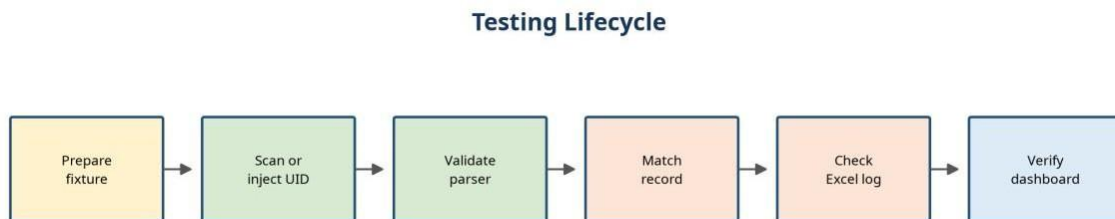


Figure 7.1 Testing lifecycle.

7.3 Functional Test Cases

Functional tests verify that each requirement produces the expected behavior.

Table 7.3 Functional Test Cases

ID	Test condition	Expected result	Status
F-01	Scan registered patient UID	Patient record displayed and event logged	Pass / To be recorded
F-02	Scan registered asset UID	Asset record displayed and event logged	Pass / To be recorded

F-03	Scan unknown UID	Exception shown; no false match	Pass / To be recorded
F-04	Repeat same tag quickly	Duplicate policy applied	Pass / To be recorded
F-05	Open event history	Recent event visible with timestamp	Pass / To be recorded
F-06	Filter by department	Only matching records displayed	Pass / To be recorded

7.4 Hardware and SPI Testing

The hardware test checks power, wiring, reset behavior, reader initialization, and tag detection. SPI wiring should be verified against the selected ESP32 board configuration. The reader should be tested with more than one tag and at more than one orientation so that the team understands the practical read zone.

7.5 Serial Communication Testing

Serial tests verify the port, baud rate, newline convention, frame format, and reconnect behavior. The parser should be tested with leading whitespace, lowercase characters, separators, empty lines, partial frames, and invalid characters.

7.6 Database and Logging Testing

Database tests verify that records can be loaded, required columns are detected, matching is deterministic, and accepted scans append exactly one event. Backup and restore tests should be completed before a demonstration.

7.7 Dashboard Testing

Dashboard tests verify that metrics update after a scan, filters behave correctly, the latest event is easy to identify, and errors are shown to the operator. The dashboard should remain usable when the workbook contains no events or only exception events.

7.8 Integration Test Summary

Integration testing follows one event from a physical tag to the dashboard. It confirms that the UID survives on each interface without

unintended transformation and that the timestamp, location, entity type, and result remain consistent.

Table 7.8 Integration Test Summary

Integration step	Verification
Reader to ESP32	UID is captured and formatted
ESP32 to serial	Frame is transmitted with newline
Serial to Python	Line is parsed and normalized
Python to workbook	Correct record and log row updated
Workbook to dashboard	Latest state and event history refresh
Exception path	Unknown or invalid input remains visible

7.9 Performance Observation

For a small workbook and local serial connection, the prototype is expected to feel responsive during the demonstration. However, exact latency and throughput depend on the host computer, serial configuration, workbook size, polling rate, and dashboard refresh behavior. A production evaluation should measure scan-to-display latency, duplicate suppression accuracy, sustained event rate, workbook write time, and recovery time after disconnection.

7.10 Reliability and Limitations

Reliability is influenced by tag placement, reader interference, USB stability, file locking, and operator procedure. Excel-based persistence is appropriate for a prototype but can become a bottleneck when many readers or concurrent users write about events. The prototype should therefore be described as proof of concept, not as a production-ready patient tracking system.

7.11 Chapter Summary

The test plan covers positive, negative, and integration paths. The most important acceptance criterion is that the system never silently presents an unknown or conflicting UID as a valid record. Future testing should use representative hardware layouts, larger datasets, multiple readers, and privacy/security review.

CHAPTER 8

APPENDICES

8.1 Introduction

The prototype establishes a foundation that can be extended in several directions. The following enhancements are prioritized according to operational value, scalability, security, and ease of validation.

8.2 Multi-Location RFID Tracking

Multiple readers can be installed at controlled checkpoints such as registration, OPD entry, diagnostics, pharmacy, ward, equipment store, and discharge. Each reader should have a unique Reader_ID and a mapped department. The system should distinguish a true movement event from repeated scans at the same checkpoint.

8.3 Cloud or Relational Database

Excel can be replaced by SQLite for a single-user local deployment or PostgreSQL/MySQL for concurrent use. A database would provide transactions, indexes, constraints, backup options, and query performance. A cloud database could support multiple departments but would require network security, authentication, outage handling, and data governance.

8.4 Automated Alerts

The system could alert staff when an oxygen cylinder is not located, an asset remains in maintenance beyond a threshold, a patient has not reached the next checkpoint within an expected interval, or an unknown tag is repeatedly detected. Alerts must be configurable and should not create alarm fatigue.

8.5 Advanced Patient-Flow Analytics

With reliable event histories, the system could estimate queue length, average dwell time, bottleneck departments, peak periods, and transition frequencies. Analytics should report data completeness and confidence because missing scans can distort the result.

8.6 Role-Based Access and Auditability

Future versions should implement role-based access for administrators, nurses, technicians, and viewers. Every change to a mapping or status should have an authenticated actor, timestamp, and reason. Patient-facing identifiers should be minimized and protected.

8.7 Networked Readers and Edge Gateways

Readers can communicate over Wi-Fi or Ethernet through an edge gateway. A message broker such as MQTT could transport scan events, while local buffering would protect against temporary network outages. The design should prevent duplicate delivery and should preserve event ordering where it matters.

8.8 Mobile and Accessible Interfaces

A responsive mobile interface can support technicians searching for equipment and scanning a tag near the point of use. Accessibility improvements may include high-contrast status indicators, keyboard navigation, clear error text, and multilingual labels.

8.9 Integration with Hospital Systems

A mature system could exchange approved operational data with an electronic health record, inventory system, maintenance system, or admission-discharge-transfer system through controlled APIs. Integration should be designed with least privilege and should avoid duplicating sensitive clinical data.

8.10 Future Development Roadmap

A practical roadmap is to first stabilize the single-reader prototype, then add a relational database and role-based access, followed by multi-location readers and alerts, then analytics and controlled integration. Each stage should have test criteria and a privacy review before operational use.

8.11 Prototype Setup Checklist

Before demonstrating the system, confirm that the RC522 reader is wired to the selected ESP32 pins, the board is powered correctly, the USB cable is connected, the serial port is identified, and the Python environment contains the required packages. Confirm that the workbook has the expected sheets and that a backup copy has been created.

Table 8.11 Prototype Setup Checklist

Check	Description	Completed
A-01	Reader wiring and common ground verified	[]

A-02	ESP32 firmware uploaded	[]
A-03	Serial port and baud rate configured	[]
A-04	Sample patient and asset mappings loaded	[]
A-05	Workbook backup created	[]
A-06	Dashboard starts without errors	[]
A-07	Known patient tag tested	[]
A-08	Known asset tag tested	[]
A-09	Unknown tag behavior tested	[]

8.12 Demonstration Dataset

The following anonymized dataset illustrates the type of records used during a classroom demonstration. Names are replaced by labels, and the values are not intended to represent real patient information.

Table 8.12 Demonstration Dataset

UID	Entity type	Entity ID	Label	Department	Status
04A1B2C3D4E5	Asset	AST-007	Wheelchair-07	OPD	In Use
0933FF10AA22	Patient	PAT-014	Patient-014	Diagnostics	Active
088877665544	Asset	AST-012	Oxygen-Cylinder-12	Equipment Store	Available
01ABCDEF2345	Asset	AST-003	Patient-Monitor-03	Ward A	Maintenance
0A12BC34DE56	Patient	PAT-021	Patient-021	Registration	Waiting

In a real system, patient identifiers should be minimized, protected, and governed by institutional policy. The demonstration dataset is deliberately limited to operational labels and should be destroyed or replaced after testing.

8.13 Sample Event Log

A scan log captures the sequence of observations that supports patient-flow and asset-tracking views. The event sequence below is illustrative and should be replaced by actual test records when the prototype is operated.

Table 8.13 Sample Event Log

Event ID	Timestamp	UID	Type	Location	Result
EVT-0001	2026-08-26 10:35:24	0A12BC34DE56	Patient	Registration	Accepted
EVT-0002	2026-08-26 10:40:06	0933FF10AA22	Patient	Diagnostics	Accepted
EVT-0003	2026-08-26 10:42:18	04A1B2C3D4E5	Asset	OPD	Accepted
EVT-0004	2026-08-26 10:43:02	01DEADBE1234	Unknown	Main Gate	Review
EVT-0005	2026-08-26 10:43:04	04A1B2C3D4E5	Asset	OPD	Duplicate

8.14 Operator User Guide

Start the ESP32 and connect it to the host computer. Open the Python processing environment and verify the serial connection. Start the Streamlit dashboard and confirm that the connection status is visible. Present a registered tag to the reader, wait for the confirmation message, and inspect the latest event card. Use the event history to verify that the timestamp and department are correct.

- If an unknown tag is presented, do not assign it informally during an active demonstration.
- Record the UID in the exception view and register it only through an authorized setup procedure.
- If the serial connection fails, check the cable, selected port, permissions, and board reset state.

8.15 Risk Assessment

A responsible deployment considers operational, technical, privacy, and safety risks.

Risk	Impact	Mitigation
Missed scan	Incomplete movement history	Use defined checkpoints and exception review
False association	Incorrect operational information	Enforce unique UID mapping and audit changes
Lost workbook	Loss of event history	Use backups and migrate to transactional storage
Unauthorized access	Privacy exposure	Authentication, least privilege, encryption
Reader interference	Unreliable identification	Site survey and controlled reader placement
Network outage	Delayed synchronization	Local buffering and retry queues
Alarm fatigue	Ignored alerts	Threshold tuning and escalation policy

8.16 Validation Protocol for Future Multi-Reader Deployment

Before installing several readers, define the reader map, expected transition paths, tag placement rules, duplicate-event policy, and offline behavior. Test each checkpoint individually before enabling the complete flow. Then conduct controlled movement scenarios with volunteers or simulated asset transfers. Compare expected sequences with recorded events and document missed, duplicated, delayed, and out-of-order observations.

8.17 Project Learning Outcomes

The project strengthened practical understanding of embedded systems, RFID communication through SPI, serial protocols, Python data processing, Excel workbook management, dashboard design, modular architecture, testing, and documentation. It also highlighted the difference between a working demonstration and a production healthcare system, particularly in relation to privacy, reliability, security, and governance.

8.18 Final Appendix Summary

The appendices provide practical material for setup, demonstration, validation, maintenance, troubleshooting, risk review, and future evaluation. They can be adapted into laboratory records or a project viva handout.

CHAPTER 9

INTRODUCTION

9.1 Project Outcome

The completed prototype connects an RC522 RFID reader, ESP32 Dev Module, serial communication, Python processing, an Excel database, and a Streamlit dashboard. A tag UID is captured, normalized, matched with a patient or asset record, stored as a timestamped scan event, and presented as a dashboard result.

9.2 Major Achievements

The project achieved hardware and software integration, SPI-based RFID communication, ESP32 firmware development, serial data transfer, Python data processing, Excel workbook management, dashboard development, scan-time logging, scan-count maintenance, and a conceptual patient-flow view.

9.3 Significance of the Developed System

The system demonstrates that a low-cost prototype can provide a common workflow for both patient movement and equipment tracking. Its modular structure supports learning and future migration to stronger databases, networked readers, and more advanced analytics.

9.4 Limitations

The prototype is not a certified medical device, does not provide clinical diagnosis, and should not be used as the sole method for patient identification or emergency decision-making. The use of Excel limits concurrent access and transaction guarantees. RFID read range, interference, tag placement, incomplete scans, privacy, and authentication also require careful attention before deployment.

9.5 Final Conclusion

The project provided practical experience in embedded systems, RFID communication, data management, and visualization. It shows how a clear event model and layered architecture can connect physical hospital operations to digital information. With multi-location tracking, cloud databases, automated alerts, strong security, and validated analytics, the prototype can be developed into a more capable hospital operations support system.

CHAPTER 10

REFERENCES

- [1] NXP Semiconductors. “MFRC522: Standard performance MIFARE and NTAG frontend.” Product documentation and datasheet.
<https://www.nxp.com/products/rfid-nfc/nfc-hf/nfcreaders/standard-performance-mifare-and-ntag-frontend:MFRC52202HN1>
- [2] Espressif Systems. “Technical Documents and ESP-IDF Programming Guides.” <https://www.espressif.com/en/support/documents/technical-documents>
- [3] Streamlit Documentation. “Streamlit documentation: an open-source Python framework for dynamic data apps.” <https://docs.streamlit.io/>
- [4] Python Software Foundation. “Python Documentation.”
<https://docs.python.org/3/>
- [5] pandas development team. “pandas documentation.”
<https://pandas.pydata.org/docs/>
- [6] openpyxl documentation. “openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files.” <https://openpyxl.readthedocs.io/>
- [7] Arduino Documentation. “SPI library and communication concepts.”
<https://docs.arduino.cc/learn/communication/spi/>
- [8] World Health Organization. “Global strategy on digital health 2020–2025.”
<https://www.who.int/publications/i/item/9789240020924>
- [9] ISO/IEC. “ISO/IEC 14443 identification cards — Contactless integrated circuit cards.” <https://www.iso.org/standard/73596.html>
- [10] OWASP Foundation. “Application Security Verification Standard.”
<https://owasp.org/www-project-application-security-verification-standard/>
- [11] National Institute of Standards and Technology. “Privacy Framework.”
<https://www.nist.gov/privacy-framework>
- [12] Project development notes and prototype test records prepared by the project team, 2026.

10.1 Reference Usage

References [1]–[3] support the principal technical descriptions of the MFRC522, ESP32 documentation ecosystem, and Streamlit. References [4]–[7] support the software development environment. References [8]–[11] provide broader context for digital health, contactless identification, and security/privacy considerations. The project development notes in [12] describe the prototype-specific design decisions and observations.

10.2 Final Note

The report describes an academic prototype and intentionally avoids presenting it as a clinically validated or production-ready hospital information system. Any future operational deployment should undergo institutional approval, privacy assessment, cybersecurity review, usability testing, and clinical/administrative validation.